

POLITECNICO
MILANO 1863

COMPUTER SCIENCE AND ENGINEERING

SOFTWARE ENGINEERING II

2025 – 2026

DD

Design Document

Best Bike Paths (BBP)

Andrea Rubagotti [286446], Maria Concetta Santagata [306018], Samuele Leanza [991114]

December 24, 2025

Version 1.0

Contents

1 INTRODUCTION **2**
- 1.1 Scope . . . 2
  - 1.1.1 Review of domain and product . . . 2
  - 1.1.2 Main architectural choices . . . 2
- 1.2 Definitions, acronyms, abbreviations . . . 2
  - 1.2.1 Definitions . . . 2
  - 1.2.2 Acronyms . . . 3
  - 1.2.3 Abbreviations . . . 3
- 1.3 Revision History . . . 3
- 1.4 Document structure . . . 3

2 ARCHITECTURAL DESIGN **4**
- 2.1 Overview: high-level components and interactions . . . 4
  - 2.1.1 System View . . . 4
- 2.2 Component view . . . 6
  - 2.2.1 3-Tier architecture . . . 6
  - 2.2.2 Microservices of the Application Tier . . . 6
  - 2.2.3 External Services (Map Service) . . . 7
- 2.3 Deployment view . . . 9
- 2.4 Component interfaces . . . 12
  - 2.4.1 External interfaces (Mobile Application → API Gateway) . . . 12
  - 2.4.2 Internal interfaces (microservice → microservice) . . . 14
  - 2.4.3 Common response codes and error handling . . . 14
- 2.5 Runtime view . . . 14
- 2.6 Selected architectural styles and patterns . . . 25
- 2.7 Other design decisions . . . 26

3 USER INTERFACE DESIGN **28**

4 REQUIREMENTS TRACEABILITY **33**

5 IMPLEMENTATION, INTEGRATION AND TEST PLAN **34**
- 5.1 Overview . . . 34
- 5.2 Implementation Plan . . . 34
- 5.3 System Testing Plan . . . 41

6 EFFORT SPENT **42**

7 REFERENCES **42**

1 INTRODUCTION

1.1 Scope

1.1.1 Review of domain and product

Cycling has become an increasingly popular activity as people become more aware of their personal health. In this reality, cyclists often rely on digital tools to track their performance, discover new routes, and share information about their conditions. Existing applications are often incomplete, as they either provide only path suggestions or they only function as forums. Best Bike Paths (BBP) addresses this gap by providing an integrated platform that supports both the recording of cycling activities and the creation of a shared, dynamic inventory of bike routes, which also supports the creation of a biking community. The system operates in a domain where accurate path information, real-time weather conditions and users' data play a crucial role in ensuring a safe and efficient cycling experience.
From a product perspective, BBP allows registered users to record trips, to obtain performance statistics (distance, average speed, minimum speed, maximum speed), and to enrich trip data through automatically detected data (by GPS, accelerometer and gyroscope). Meteorological information are retrieved from external services and are useful for enriching trips' data. Users can also provide manual inputs about bike paths, such as surface conditions or the presence of obstacles and they can decide whether to make this information public or not.
Both registered and unregistered users benefit from the shared repository of bike paths. The system computes and displays optimal routes between a chosen origin and destination, ranking them thanks to a score, which is calculated according to path quality, efficiency and overall usability. When multiple users contribute data about the same path, BBP merges the information by considering both freshness and number of confirming reports, in order to minimize false detections and to guarantee a high quality of service.
Overall, BBP combines cycling monitoring, collaborative path detection and environmental data integration to offer a complete tool for cyclists. The system keeps track of individual improvements and contributes to create a continuously updated and community-driven database of bike paths.

1.1.2 Main architectural choices

The Best Bike Paths system is based on a classic three-tier architecture, consisting of a presentation tier (mobile application), an application tier (backend services exposing REST APIs) and a data tier (persistent storage for users, trips, paths and sensor data).
Inside of the application tier, the system is organized following a microservice-oriented decomposition: user and authentication management, account management, trip recording and publication, path and condition management and path research. This structure promotes a clear separation of functions and supports an independent evolution of each service, providing a solid foundation for future scalability and maintainability.

1.2 Definitions, acronyms, abbreviations

1.2.1 Definitions

- **Registered user**: a person who has successfully created an account on the Best Bike Paths platform.
- **Non-registered user**: a person who hasn't completed the procedure of registration on Best Bike Paths platform. He can visualize suggested paths from one point to another and also path information.

- **Cycling paths/tracks**: places used exclusively for cycling or at most where cars are rare and where they cannot go over a speed limit.
- **Trip**: single cycling activity recorded by a registered user, containing data such as distance, duration, speed, and path conditions.
- **Path condition**: current state of a certain bike path. It represents its overall state, the eventual presence of potholes and its maintenance needs.
- **Score**: numeric value which is assigned by the system to each path. It represents the quality and the efficiency of the path, and it is based also on user-provided data.
- **Sensor-data/automatically detected data**: data automatically acquired by the system, such as average speed, position or presence of path discrepancies by using different sensors, such as GPS, accelerometer and gyroscope. This feature is useful not only for tracking path conditions, but also for evaluating users' performance.
- **External service**: a third party system used by Best Bike Paths, which gives information about meteorological conditions when going on a trip.
- **Meteorological information**: this is environmental data, useful when going on a specific path.

1.2.2 Acronyms

- **BBP**: Best Bike Paths

1.2.3 Abbreviations

- **sd**: sequence diagram

1.3 Revision History

- Version 1.0: December 24, 2025

1.4 Document structure

- **INTRODUCTION**: this section provides a brief description of the project, focusing on the goals to be achieved, as described in the RASD.
- **ARCHITECTURAL DESIGN**: this section describes the system architecture at different levels of granularity.
- **USER INTERFACE DESIGN**: this section defines the platform's user interface.
- **REQUIREMENTS TRACEABILITY**: this section describes how the goals outlined in the RASD are fulfilled by mapping the requirements to the components presented here.
- **IMPLEMENTATION, INTEGRATION AND TEST PLAN**: this section outlines the implementation, integration, and testing activities, offering support for the platform's developers and maintainers.
- **EFFORT SPENT**: this section reports the effort dedicated to producing this document.
- **REFERENCES**: this section includes references to the documents and software used for the preparation of this document.

2 ARCHITECTURAL DESIGN

2.1 Overview: high-level components and interactions

This section provides a high-level overview of the system's architecture, detailing its main components and the interactions among the different architectural tiers. It explains the design decisions adopted to satisfy both the functional and non-functional requirements explained in the RASD. The description clarifies how the system is structured, how responsibilities are distributed across components and how architectural choices support performance, scalability, reliability and overall system quality.

2.1.1 System View

To support BBP capabilities, the chosen architectural design is a 3-tier architecture. This structure consists of one tier for the presentation layer, one for the application layer and one for the data layer:

- **Presentation Tier (Client-side)**: this layer is responsible for displaying the user interface through the mobile application. It handles the interaction with users and the acquisition of sensor data from the device.
- **Application Tier (Server-side)**: this layer comprehends the core server-side logic. It includes components for handling requests through an API Gateway, which then routes requests to the corresponding microservice, such as the logic for merging conflicting path data or retrieving path information. These components work together within the same application tier.
- **Data Tier**: this layer includes the databases and the DBMS, which are necessary for persistent storage.

Information flow within the system moves from the front-end to the back-end: starting from the presentation layer where users interact with the system, through the API gateway in the application layer, to the specific microservices that interpret requests and finally to the data layer. Once the data is retrieved and processed, it is formatted and conveyed back to the presentation layer.
As the main architectural style for the application tier, the system uses microservices. This approach consists of the implementation and integration of many small, independent services that communicate through REST API calls, providing a flexible solution for a system which is expected to rapidly scale. Both the application and data layers are thought to be replicated over different machines in order to ensure redundancy, ensuring high availability and fault tolerance.

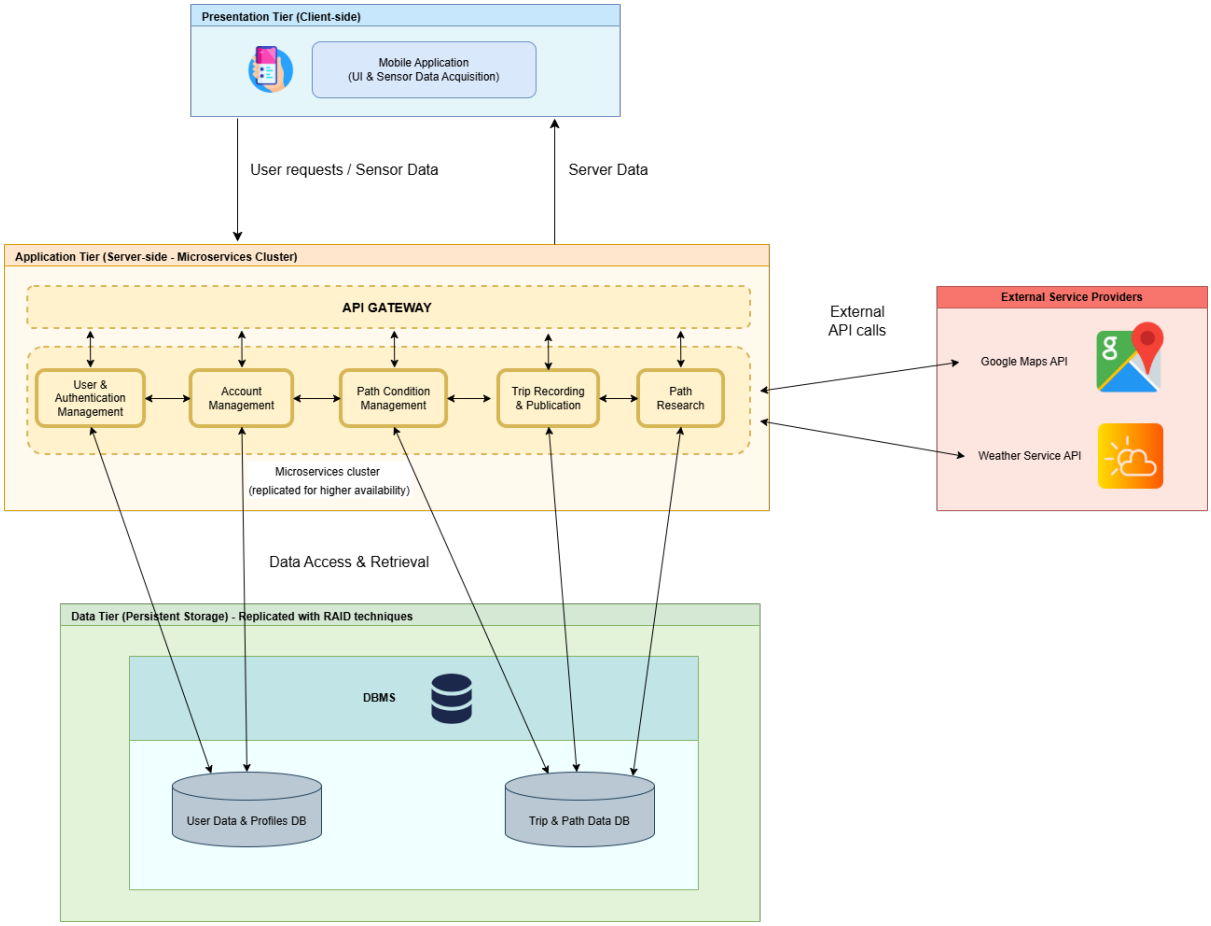


Figure 1: Three-tier architecture with microservices

2.2 Component view

2.2.1 3-Tier architecture

Presentation Tier

The presentation tier is the point of interaction between users and the Best Bike Paths platform. Users access the system through a mobile application, which is responsible for displaying information about trips, bike paths and user's performance. This layer manages all the user-interface elements, including map visualization, forms compilation for manual data entry, trip management interfaces and route search functionalities. Its basic role is to acquire user input, translate it into requests to the application layer and display the responses received from the backend services in a human comprehensible way. By isolating the user interface from the system's internal logic, this tier ensures a clear separation of concerns and an intuitive navigation in all the application's sections.

Application Tier

The application tier constitutes the core of the system: it handles all business logic and coordinates the flow of information between presentation layer and data one. It is organized into modular services (deployable as microservices) which manage key functionalities. These are explained in section 2.2.2 (Microservices of the Application Tier). Each service exposes APIs that allow the platform to do requests processing, input validation, statistics computing, data aggregation and to allow interaction with external providers. Communication between internal services occurs through RESTful calls, ensuring clear interfaces and extensibility. Furthermore, this tier is deployed in multiple replicated instances to ensure redundancy, high availability, and continuous operation even during single failures. A load balancer distributes incoming requests across the available instances of the Application Tier, ensuring efficient resource utilization, preventing overload. This is essential to maintain consistent performance under varying traffic conditions. Moreover, a circuit breaker mechanism is implemented to detect failing services and temporarily isolate them, preventing unnecessary cascading faults and the emergence of new errors.

Data Tier

The data tier is responsible for reliable and efficient storage of all information managed by the system. It includes persistent repositories containing user profiles, trip records, GPS traces, sensor-data, bike path descriptions, user-submitted conditions and problems, together with cached meteorological data. To prevent bottlenecks or security lacks and in order to improve scalability, the data tier is organized into two separate database systems (both managed by the same DBMS):

- a dedicated database for user-related information, including user profiles, authentication data and personalized settings;
- a second database for operational data, such as trip records, path descriptions and their aggregated conditions, GPS tracks and problems.

Each dataset is structured to support the needs of the corresponding application services, ensuring fast data creation and retrieval and consistent updates. This tier guarantees data integrity and fully supports concurrency, to guarantee the continuity of the service, even during peak workloads.

2.2.2 Microservices of the Application Tier

The main functionalities of the Application Tier are organized into the following microservices:

User & Authentication Management

This microservice is responsible for handling user registration, login, logout, credential management and token or session issuance and invalidation. It can access the user-related database to read and update authentication data, user identities and profile details; instead, it does not directly access operational trip or path data.

Account Management

This microservice manages user profiles, personal information, preferences, privacy settings and account life-cycle operations (such as profile updates). It interacts only with the user-related database, which stores profile attributes and configuration options. In addition, this microservice manages *privacy zones*, which basically are user-defined sensitive areas (e.g., home, workplace) that must not be disclosed through published location data. Privacy zones are stored as geofences (circular areas) or polygons associated with the user profile and are later enforced during trip publication and path sharing.

Trip Recording & Publication

This microservice handles the creation, update, storage, and publication of user trips, including GPS traces, timestamps, and associated data. It writes and reads data from the operational database, where trip records and related geo-spatial information are stored. User information is referenced via hidden identifiers (e.g., user IDs) rather than direct accessing to the user-related database. In fact, all the other microservices rely on the *User & Authentication Management* service to validate permissions and identities. During trip publication, this microservice enforces *privacy zones* defined by the user. In particular, when a trip (or a derived path) is published, GPS traces are sanitized server-side by masking the starting or arrival points that fall within user's privacy zone (for example houses or work places, by applying spatial generalization). This prevents the disclosure of sensitive locations while still allowing the publication of meaningful routes and aggregated path information.

Path & Condition Management

This microservice manages all the operations related to the catalog of bike paths and streets and to the dynamic conditions reported by users or derived from sensors (such as surface quality or obstacles). It has access to the operational database, which stores path tracks, users' reports and aggregated status over time. Thus, it is responsible for ensuring consistency when merging multiple contributions that refer to the same path street, handling all the computation in order to determine the correct aggregate status. Moreover, this microservice implements an *fix* policy for obstacles: an obstacle is marked as resolved when a user affirms that a path has an optimal status and his path information update has a greater timestamp than the other ones.

Path Research

This microservice provides search, filtering and ranking over the available path, supporting queries such as finding suitable routes between certain starting points and a destinations. It reads data from the operational database to retrieve all disposable paths between a starting point and a destination, ranked by their score.

2.2.3 External Services (Map Service)

Google Maps API

The BBP platform integrates the Google Maps API primarily to check bicycle-specific routes, automatically excluding infrastructures that are not accessible by bike, such as highways or high-

speed or high-trafficked roads. This functionality allows BBP to reconstruct and validate paths manually entered by users, as well as to correctly align GPS traces acquired during automated trip recording, ensuring that the resulting routes are realistic and consistent with the available cycling network. By relying on a widely adopted and reliable mapping service, BBP also provides route visualization on an interactive, detailed and fully zoomable map, guaranteeing accurate cartographic information and an improved navigation experience. Communication with the Google Maps API occurs exclusively through the *Path & Condition Management, Trip Recording & Publication* and *Path Research* microservices.

Weather.com API (Meteorological Service)

To enrich trip records and improve the completeness of path information, BBP integrates meteorological data retrieved from *Weather.com*, a trusted and reliable external meteorological service. When users record a trip, the system queries the external weather service to obtain weather information such as temperature, precipitation, visibility, humidity and wind speed. These information are stored together with the trip and can be displayed to users as part of their performance statistics. The integration is handled by the *Trip Recording & Publication* microservice, which retrieves the external data only when needed, ensuring efficient use of external resources and consistent trip enrichment.

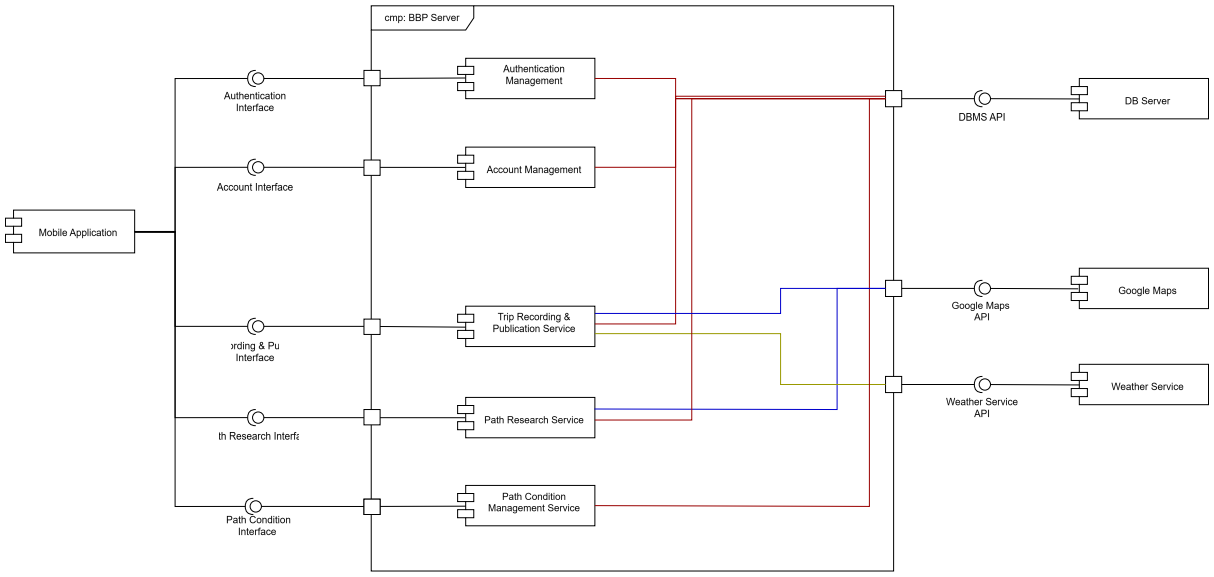


Figure 2: Component Diagram

2.3 Deployment view

This section presents the deployment view of the Best Bike Paths (BBP) system, describing the execution environment and the distribution of the hardware and software components involved. The deployment model follows a three-tier architecture, including a client-side mobile application, a replicated application tier hosting the microservices and a data tier providing persistent storage with fault tolerance (thanks to RAID systems). Additional external services are integrated to enrich functionality and support path visualization on real time maps and meteorological data retrieval.
Further details about the elements in the deployment diagram are provided below.

Mobile Application

The mobile application, installed on the user's smartphone, constitutes the main access point to the BBP platform. It handles user interactions, displays trips' information and bike paths and collects sensor data (such as GPS traces and accelerometer and gyroscope readings) during automated trip recording. Communication with the backend occurs over secure HTTPS connections, which are managed through protective and balancing components before reaching the core system.

Firewall

The firewall filters and monitors all incoming connections from mobile clients, blocking malicious traffic before it reaches the internal components. It is located at the boundary of the system, ensuring that only legitimate and safe requests can access the protected network, establishing a secure perimeter around the BBP infrastructure. A second firewall layer is also placed between the Application Tier and the external services it interacts with, such as the Google Maps API and the Weather.com meteorological service. This additional protection ensures that outbound requests and returned responses are checked, preventing the application servers from directly exposing themselves to untrusted external networks and further strengthening the overall security of the system.

Load Balancer

The load balancer distributes incoming authenticated requests across multiple instances of the Application Tier. This mechanism improves performance, avoids overload on individual servers and enhances the overall scalability and availability of the system. By dynamically selecting the most suitable application instance, it ensures faster response times and operational stability even under high user demand.

Application Servers (Microservices Cluster)

The Application Tier consists of several replicated application servers, each hosting the full set of BBP microservices. These include user authentication, account management, trip recording and publication, path condition management and path research. The microservices communicate via internal REST APIs and are responsible for coordinating business logic, validating permissions, enriching trip data, aggregating user reports, executing path search and path rating. Only this tier is allowed to communicate directly with the data tier, ensuring a very controlled access to stored information. The replication of the servers provides redundancy, fault tolerance, and uninterrupted availability during maintenance or node failures. Privacy enforcement (masking GPS traces inside user-defined privacy zones) is performed within the backend microservices before any trip or path data is published, reducing the risk of publishing sensitive locations from client-side data.

Database Servers

The Data Tier is composed of two distinct database systems, each deployed on a dedicated server and replicated for high availability. The *User-Related Database* stores authentication credentials, user identities, and profile configurations, ensuring fast and secure access for the services that handle account and identity management. The *Operational Database* contains trip records, GPS traces, path descriptions, obstacles, user-reported conditions, aggregated path status, and cached meteorological data. Each microservice interacts only with the appropriate database, maintaining a clear separation of each field and improving performance through specialized data organization.

External Services

BBP relies on two main external services to improve system functionality. The Google Maps API provides detailed and zoomable interactive maps, used for route visualization and geospatial queries needed by the Path Research microservice. Moreover, thanks to this service, BBP can check whether a path is proper for a bike ride or not. The Weather.com API supplies real-time meteorological data, which enriches trip records and supports the interpretation of sensor anomalies detected during automated recording. Communication with external services occurs exclusively through the dedicated backend microservices, ensuring controlled and secure integrations.

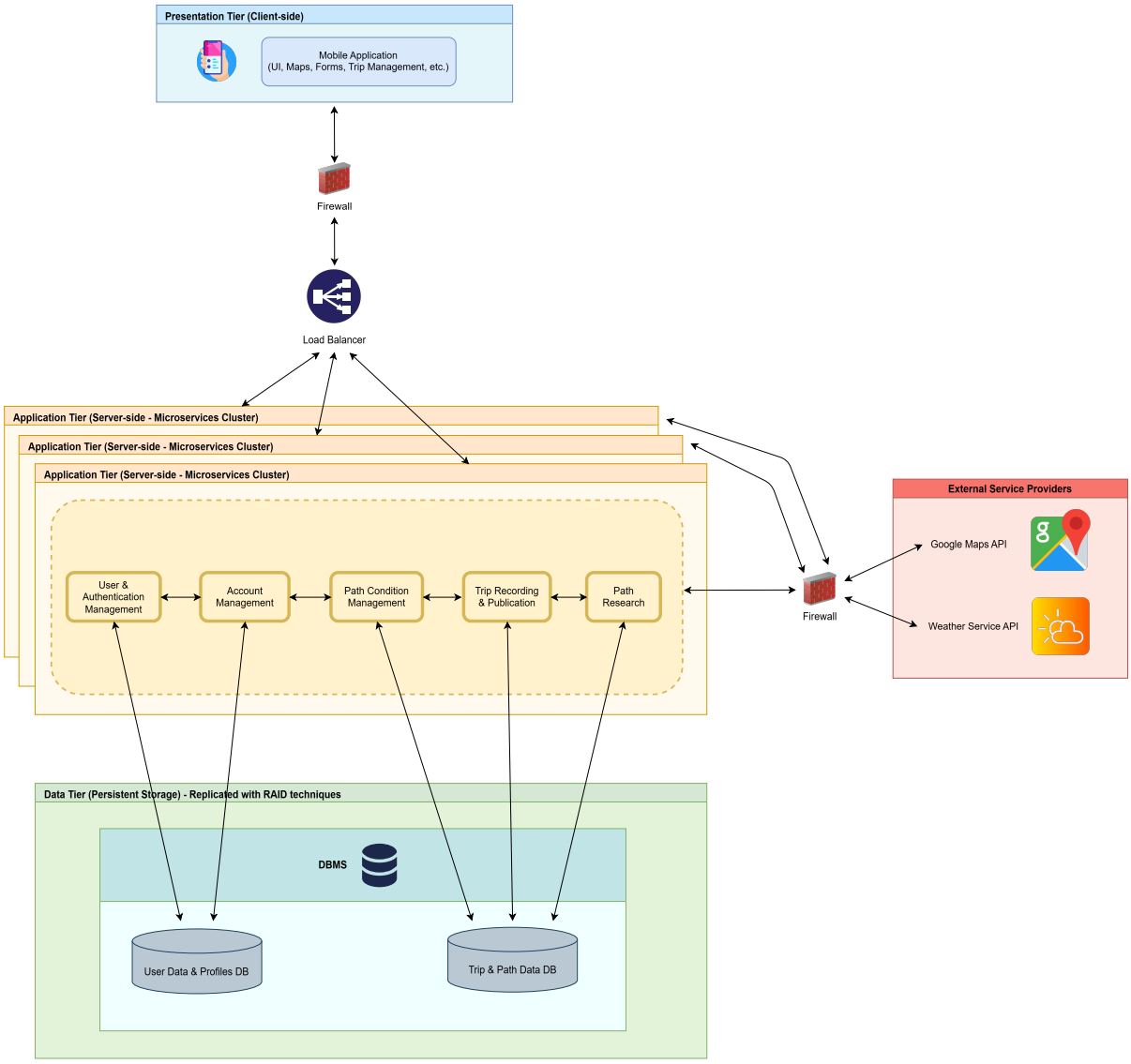


Figure 3: Deployment view

2.4 Component interfaces

This section provides a concise overview of the interfaces exposed by the BBP system and of the main methods offered by all its components. All internal components interact through RESTful APIs, ensuring clear separation of each role and enabling a complete bidirectional communication among microservices when needed. For consistency, the listed operations follow a conventional naming scheme aligned with the corresponding HTTP rules (e.g., `GET`, `POST`, `PUT`, `DELETE`) and are grouped by the microservice that provides them.
The data structures referenced as input/output parameters are named to be self-explanatory. To avoid ambiguity, throughout this document the following conventions are adopted:

- **[var?]**: a variable written in this way, means that this data is not compulsory, and it can be also a `null` value.

2.4.1 External interfaces (Mobile Application → API Gateway)

The following list summarizes the main endpoints exposed to the client. Some endpoints return JSON payloads, to simplify data exchange. Endpoints requiring authentication use a masked token: JWT access token (signed), short-lived. The JWT access token enables stateless authentication and authorization across backend microservices and it is included in the `Authorization` header.

User & Authentication Management

- `POST_createAccount(name, surname, email, password, mainRidingArea, [age?], [height?], [weight?], [sex?])`: creates a new user account and stores the user as a registered user.
- `POST_login(email, password)`: authenticates the user and returns an access token.
- `POST_logout(userId)`: terminates the active session and invalidates the token.
- `GET_validateToken()`: verifies token validity and user authorization. It's useful to understand wether a user is already logged-in or not.

Account Management

- `GET_getUserProfile(userId)`: retrieves personal data and user preferences.
- `PUT_updateUserProfile(userId, name, surname, email, age, weight, height, sex, mainRidingArea)`: updates user profile information.
- `GET_getPrivacyZones(userId)`: retrieves inferred privacy zones computed by the system, which automatically detects whenever a user starts or arrives multiple time at the very same point.

Trip Recording & Publication

- `POST_startAutomaticRegistration(userId)`: notifies the server that the user has started an automatic trip recording.
- `POST_startManualRegistration(userId, origin, destination, listOfStreets, pathCondition)`: starts a manual trip insertion.
- `POST_stopAutomaticRegistration(tripId)`: stops the trip recording and triggers statistics computation.

- `POST_uploadTripSamples(tripId, gpsPoints, sensorData)`: uploads buffered GPS and sensor samples, so the server can analyze them.
- `POST_confirmDetectedProblem(tripId, problem, confirmation)`: confirms or rejects automatically detected obstacles.
- `GET_getTripHistory(userId)`: retrieves the list of recorded trips of a certain user (returns a list of tripIds).
- `GET_getTripDetails(tripId)`: retrieves detailed trip information of a certain trip including statistics and related weather data.
- `POST_publishPathInfo(tripId)`: publishes the path information and makes it available to the community. once published, it can't return private.

Path Condition Management

- `POST_createPath(userId, streetNames, status)`: inserts a new path into the DB.
- `POST_reportPathProblem(pathId, problemType, location, severity)`: saves a new obstacle or issue. This function is called after `POST_publishPathInfo(tripId)`.
- `GET_getPathDetails(pathId)`: retrieves aggregated path information, status, score, origin, destination.
- `GET_getActiveProblems(pathId)`: retrieves active problems for a path.

Path Research

- `GET_searchPaths(origin, destination)`: retrieves all bike-compatible paths between two locations. Paths are returned in a list containing the paths with all their details and their computed scores. The ones which include high-trafficked roads or highways are not included into return list.

Google Maps API (External Map Service)

- `GET_evaluateBikeRoute(origin, destination, listOfStreets)`: returns a boolean value, 1 if the track is suitable for a bike, 0 otherwise.
- `POST_fromPointsToRoads(gpsPoints)`: aligns raw GPS samples to the most likely road/bike-network geometry, improving trace consistency and correcting sensor noise.
- `GET_geocode(address)`: converts a textual address into geographic coordinates (latitude, longitude).
- `GET_reverseGeocode(latitude, longitude)`: converts geographic coordinates into a human-readable location (easier for humans to read and understand).
- `GET_getLength(latitudeStart, longitudeStart, streetStart, latitudeEnd, longitudeEnd, streetEnd)`: returns the distance between two points on the map, considering the streets on which the points are placed onto.

Weather.com API (External Meteorological Service)

- `GET_getCurrentWeather(latitude, longitude)`: returns current weather conditions at a given location (temperature, precipitation, humidity, wind).

2.4.2 Internal interfaces (microservice → microservice)

Internal calls are executed within the Application Tier, hidden from the client. The most relevant interactions are:

- **Trip Recording & Publication → User & Authentication Management**: token validation and user authorization for trip operations. This is fundamental to ensure privacy.
- **Trip Recording & Publication → Account Management**: retrieval of privacy zones to sanitize GPS traces during publication.
- **Path Research → Path & Condition Management**: retrieval of detailed path information, such as path segments, scores and aggregated conditions needed for their ranking.
- **Path & Condition Management → Trip Recording & Publication**: retrieval of recent published path information used to confirm or invalidate old problems.

In other scenarios, the internal interfaces are not interactions between two different microservices, but are only some functions which are triggered when a certain event happens. For example:

- `POST_computeAggStatus(pathId)`: when new information are available for a path, a new aggregate status is computed.
- `POST_computeScore(pathId)`: when the aggregate status of a path is modified, this function is triggered, and a new score is computed for that path.

2.4.3 Common response codes and error handling

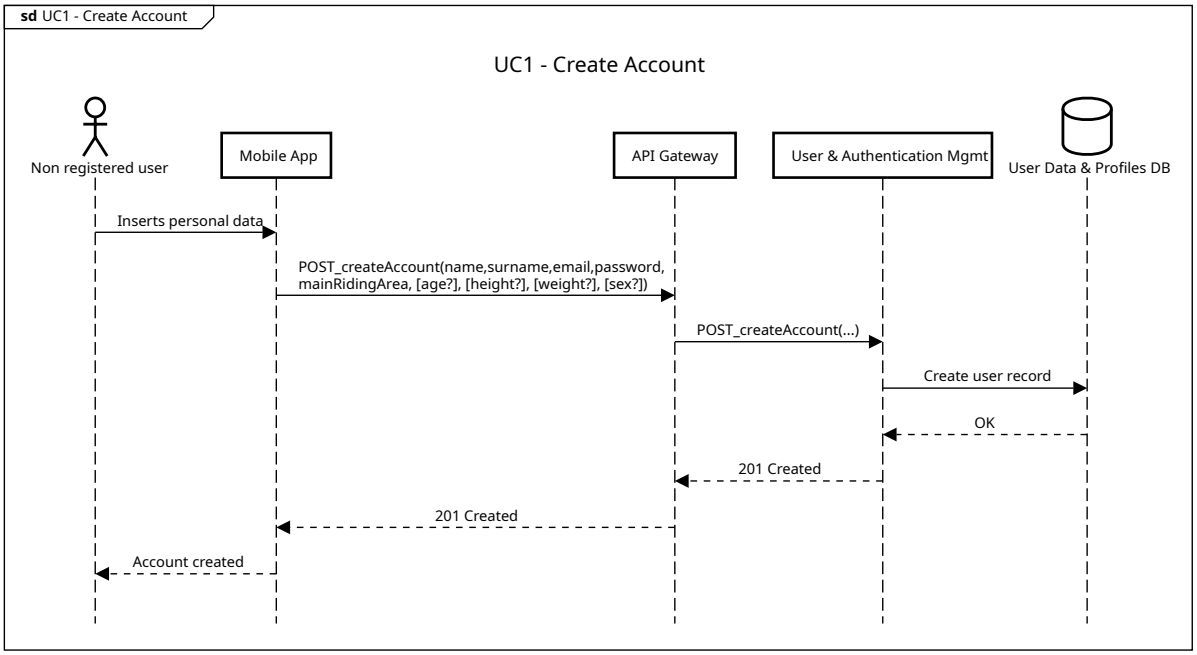
All services adopt consistent error handling through standard HTTP status codes:

- **200/201**: request successfully processed (resource retrieved/created).
- **400**: invalid input or malformed request (validation error).
- **401/403**: authentication required or insufficient permissions.
- **404**: requested resource not found.
- **409**: conflict (e.g., duplicated resource or concurrent update conflict).
- **5xx**: unexpected server error; circuit breaker and retries may apply for temporary failures.

2.5 Runtime view

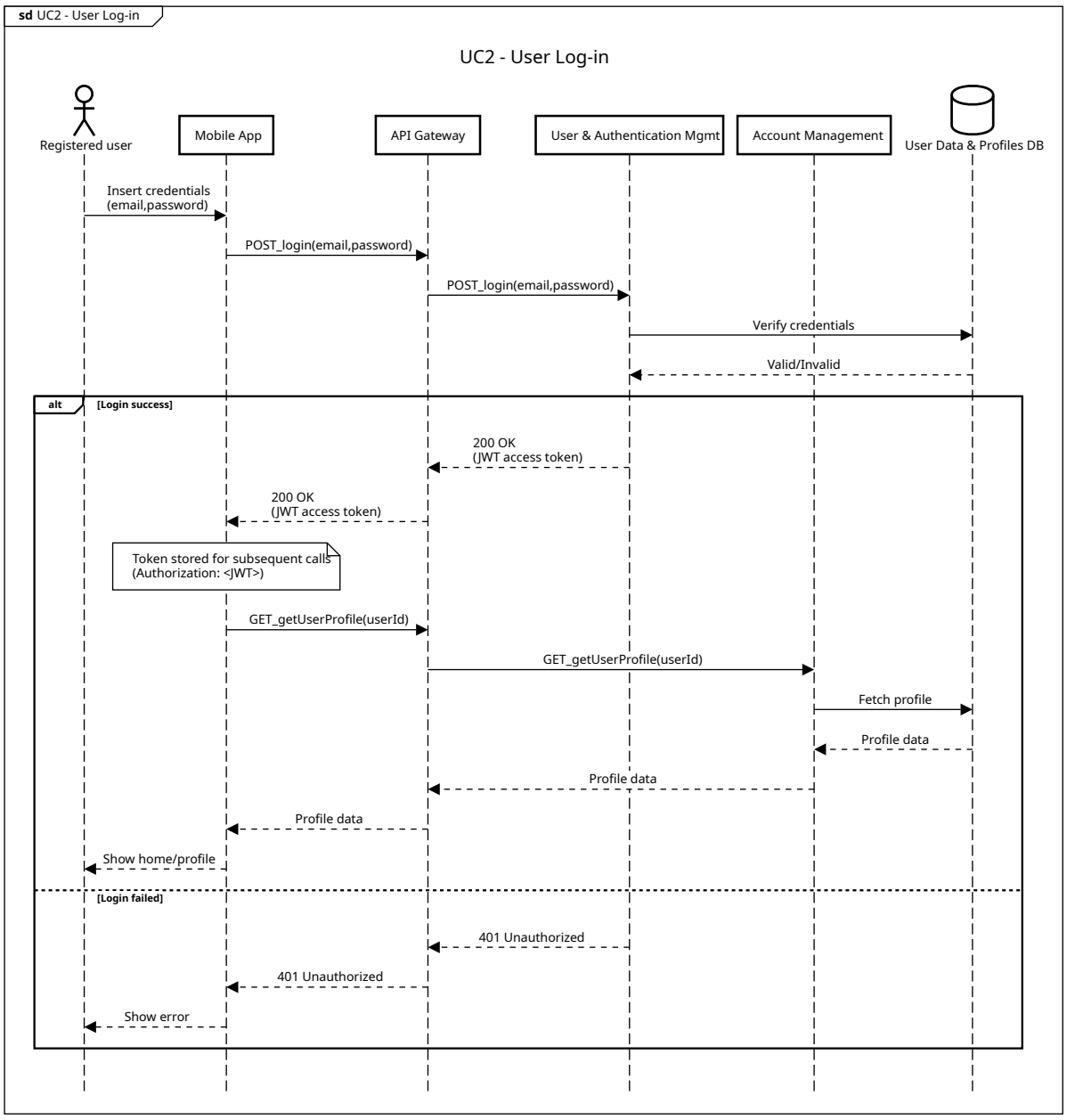
In this section, we describe the dynamic behavior of the Best Bike Paths (BBP) system by illustrating the flow of messages exchanged among the involved components through sequence diagrams, one for each relevant use case also described in the RASD document.
In order to ensure a major clarity of this document, some elements have been simplified or intentionally abstracted. In particular, authorization and authentication checks (the verification of user permissions or token validity) are not explicitly represented in the diagrams, even though they are assumed to be performed whenever required.

[UC1] - Create Account



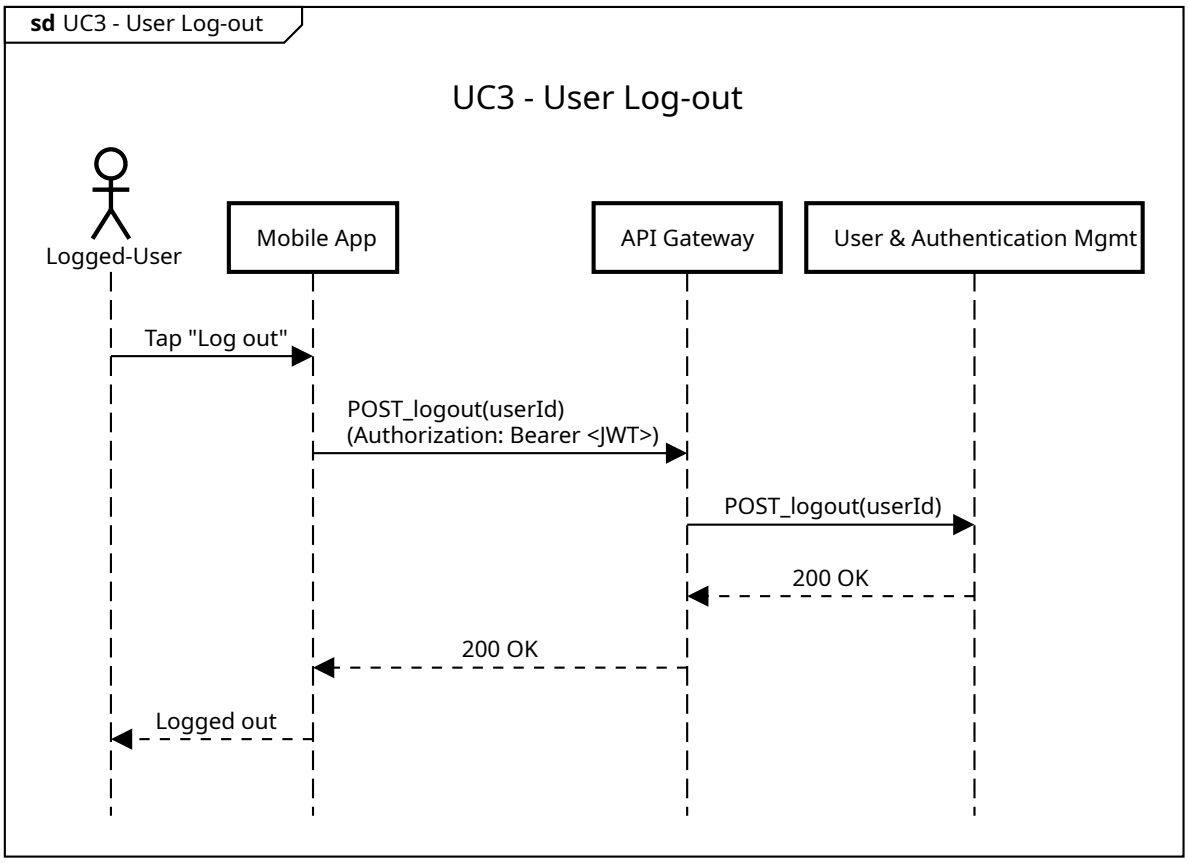
This sequence illustrates the registration process for a new user. The non-registered user enters personal data in the Mobile Application, which sends a `POST_createAccount` request to the API Gateway. The User & Authentication Management microservice validates the input (e.g., email uniqueness) and persists the new profile in the User Data & Profiles DB.

[UC2] - User Log-in



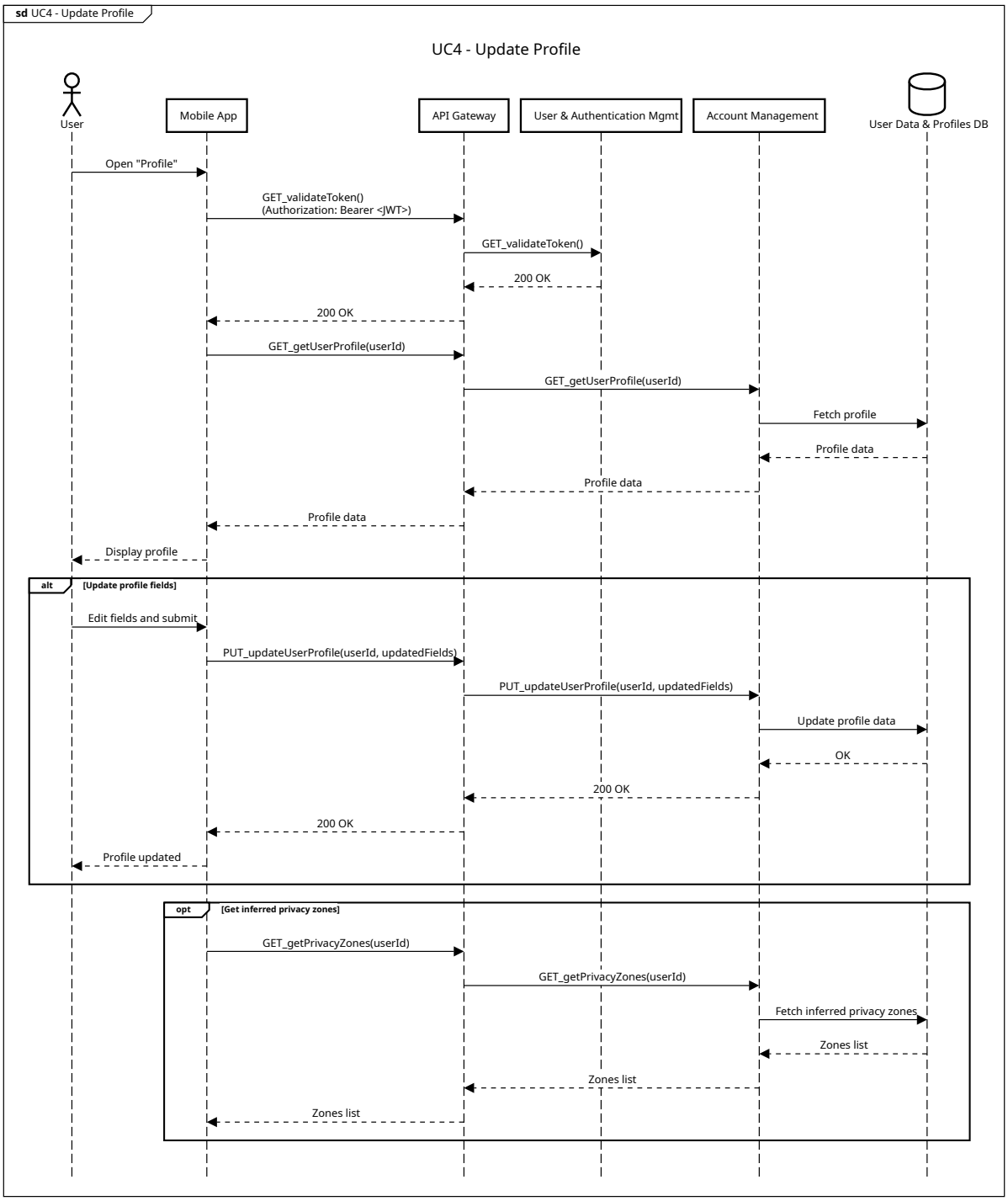
This diagram describes the authentication process for registered users. The user submits credentials, which are forwarded by the API Gateway to the User & Authentication Management microservice for verification. If they are correct, the system generates a JWT access token and then retrieves the user profile through the Account Management microservice to redirect the user to their profile page.

[UC3] - Log-out



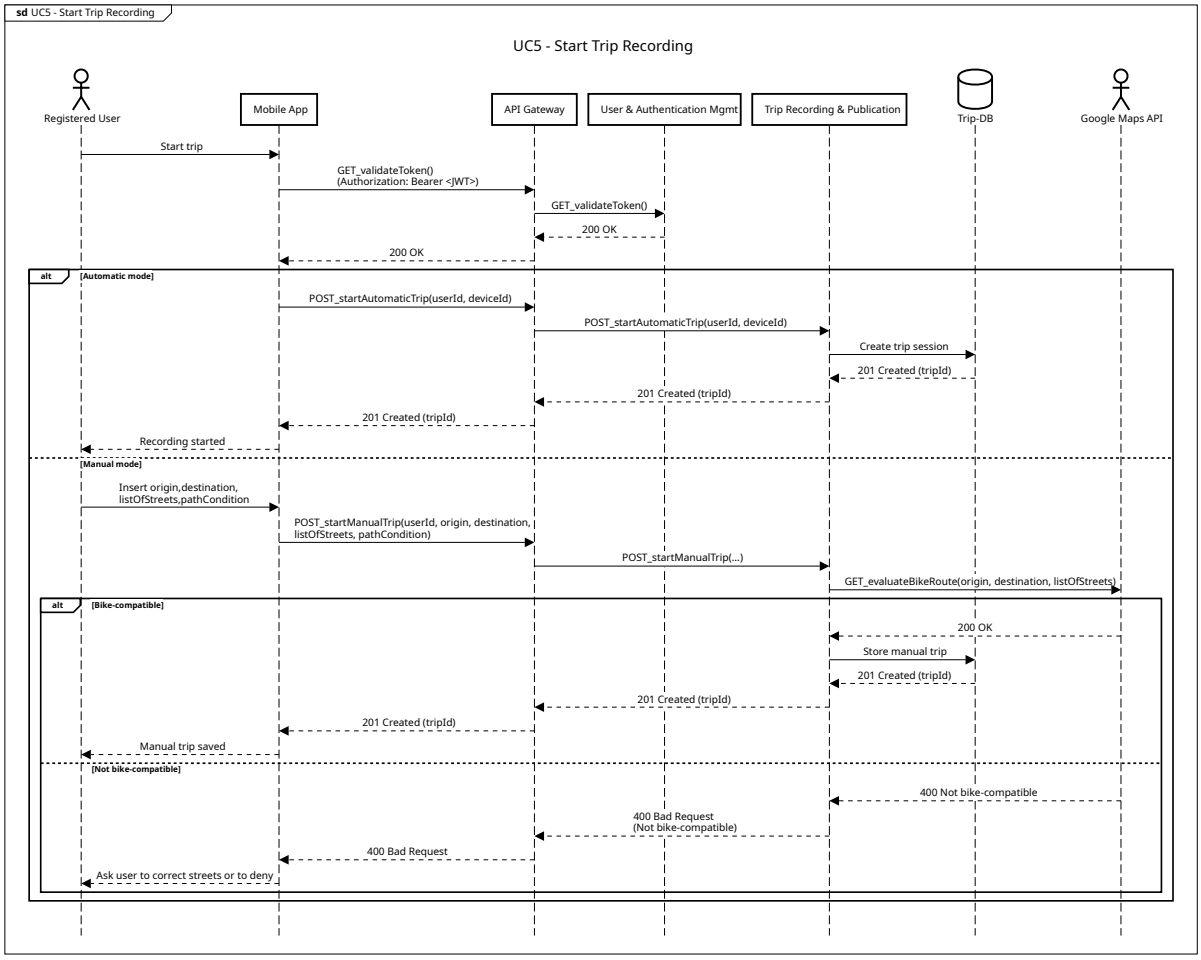
This sequence handles the secure termination of the user session. The Mobile Application sends a `POST_logout` request to the API Gateway, which forwards it to the User & Authentication Management microservice. The token is invalidated server-side and the user is returned to the login screen, preventing access to registered-only functionalities.

[UC4] - Update Profile



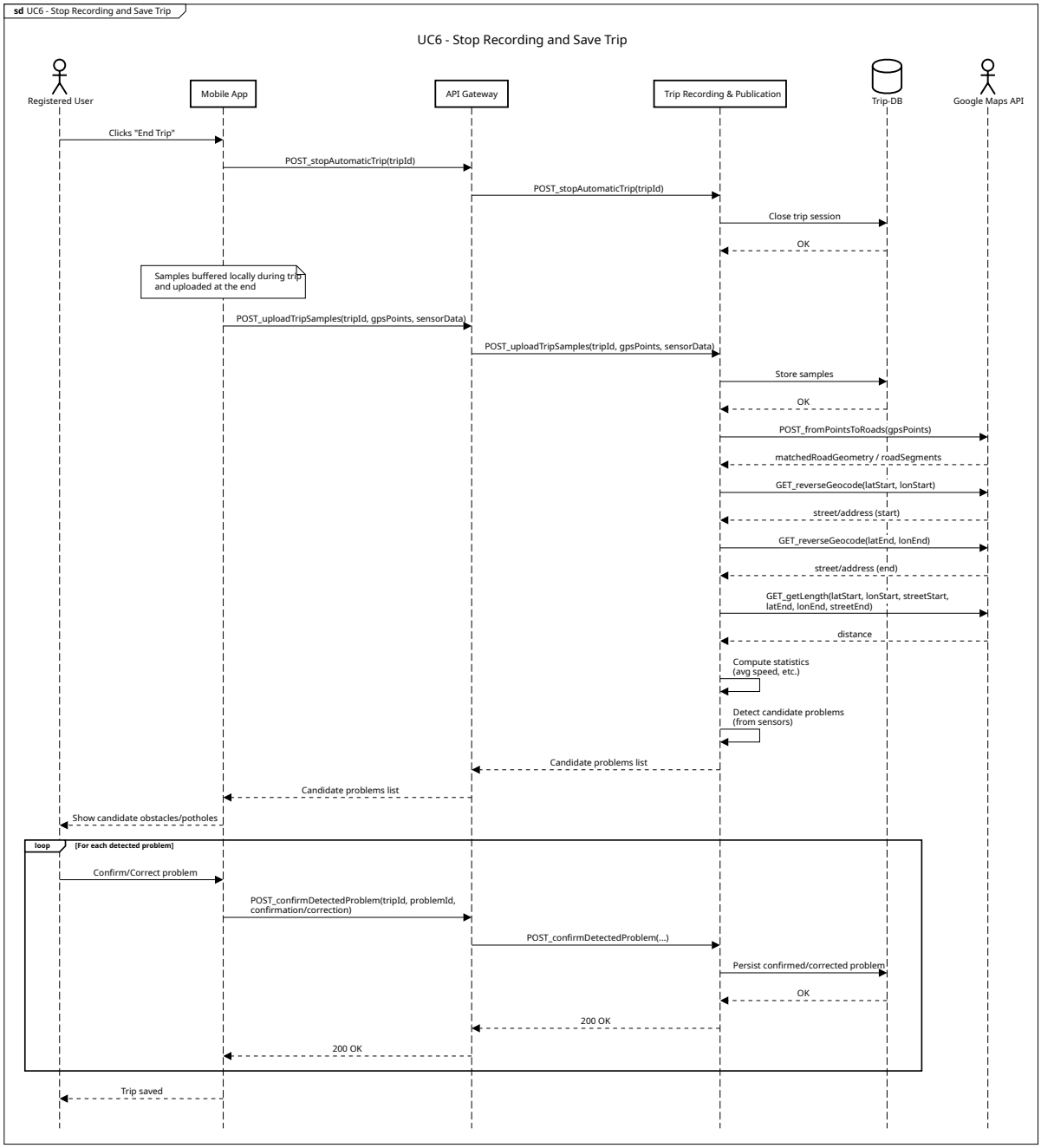
This diagram shows how a user can modify personal information. After token validation, the Account Management microservice fetches the current profile data from the User Data & Profiles DB and returns it to the Mobile Application. Once changes are submitted, the system validates updated fields and persists them in the database. The application can also check the inferred privacy zones associated with their profile.

[UC5] - Start Trip Recording



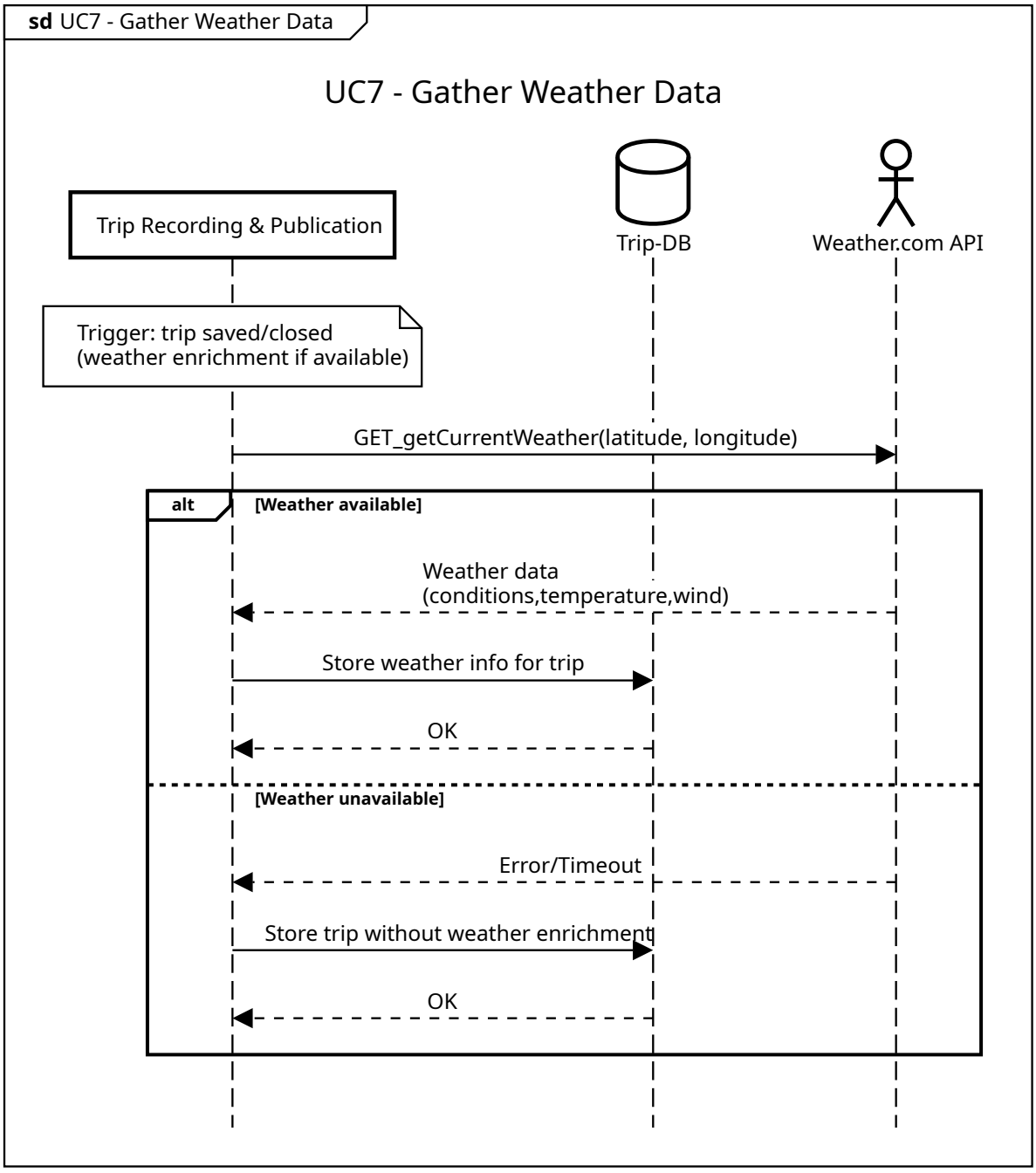
This sequence describes the start of a trip session in automatic or manual mode. In automatic mode, the Mobile Application starts sensor acquisition and notifies the Trip Recording & Publication microservice via `POST_startAutomaticTrip`, which initializes a new trip record. In manual mode, the user provides origin, destination and the list of streets, and the system checks bike compatibility before storing the manual trip.

[UC6] - Stop the Recording and Save Trip



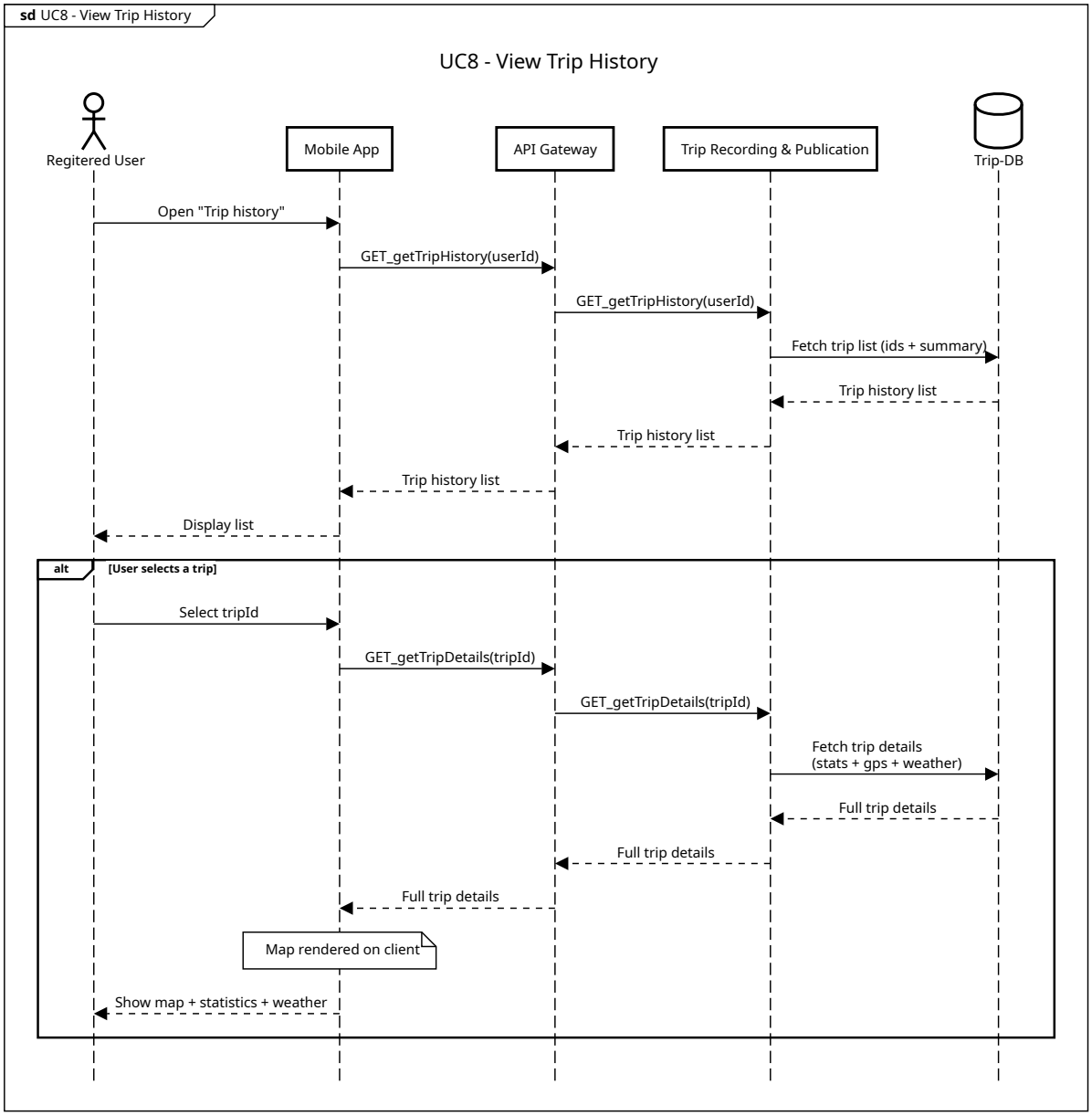
This diagram illustrates the conclusion of a trip and data persistence. When the user ends the trip, the Mobile Application stops the recording and uploads the buffered GPS and sensor samples via `POST_uploadTripSamples`. The Trip Recording & Publication microservice performs map-matching through the Google Maps API, derives the traveled distance (and related statistics). The user confirms or dismisses detected problems before the trip and its associated data are permanently stored in the Trip & Path Data DB.

[UC7] - Gather Weather Data



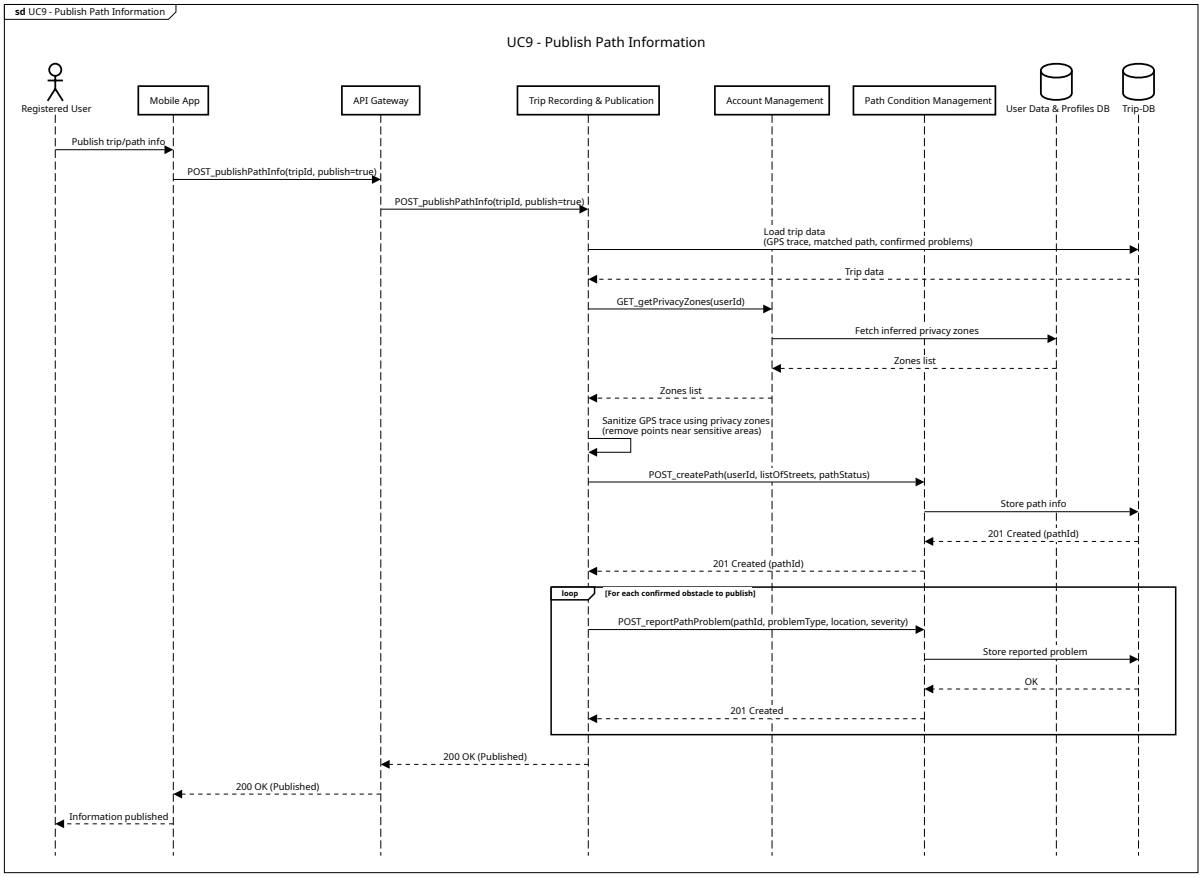
This is a system-driven process triggered after a trip is saved. The Trip Recording & Publication microservice uses the trip coordinates to query the external Weather Service through `GET_getCurrentWeather`. When available, meteorological data (temperature, wind and conditions) are associated with the trip to enrich the user's history; otherwise, the trip is stored without enrichment.

[UC8] - View Trip History



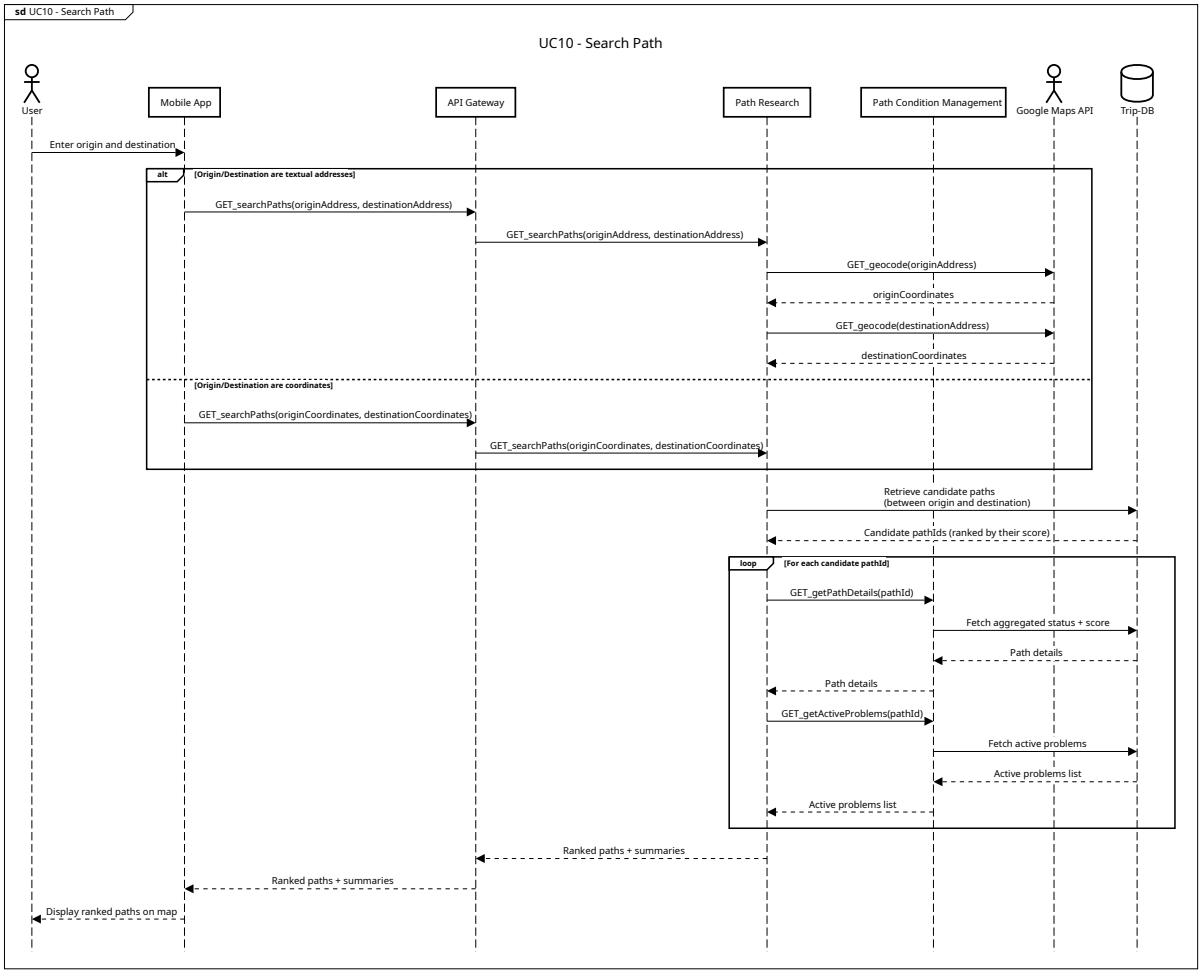
This diagram describes the consultation of past activities. The user requests the trip history and the system retrieves a list of trips from the Trip & Path Data DB. Upon selecting a single trip, the Mobile Application requests full details via `GET_getTripDetails`, including statistics and route information that is visualized on the map.

[UC9] - Publish Path Information



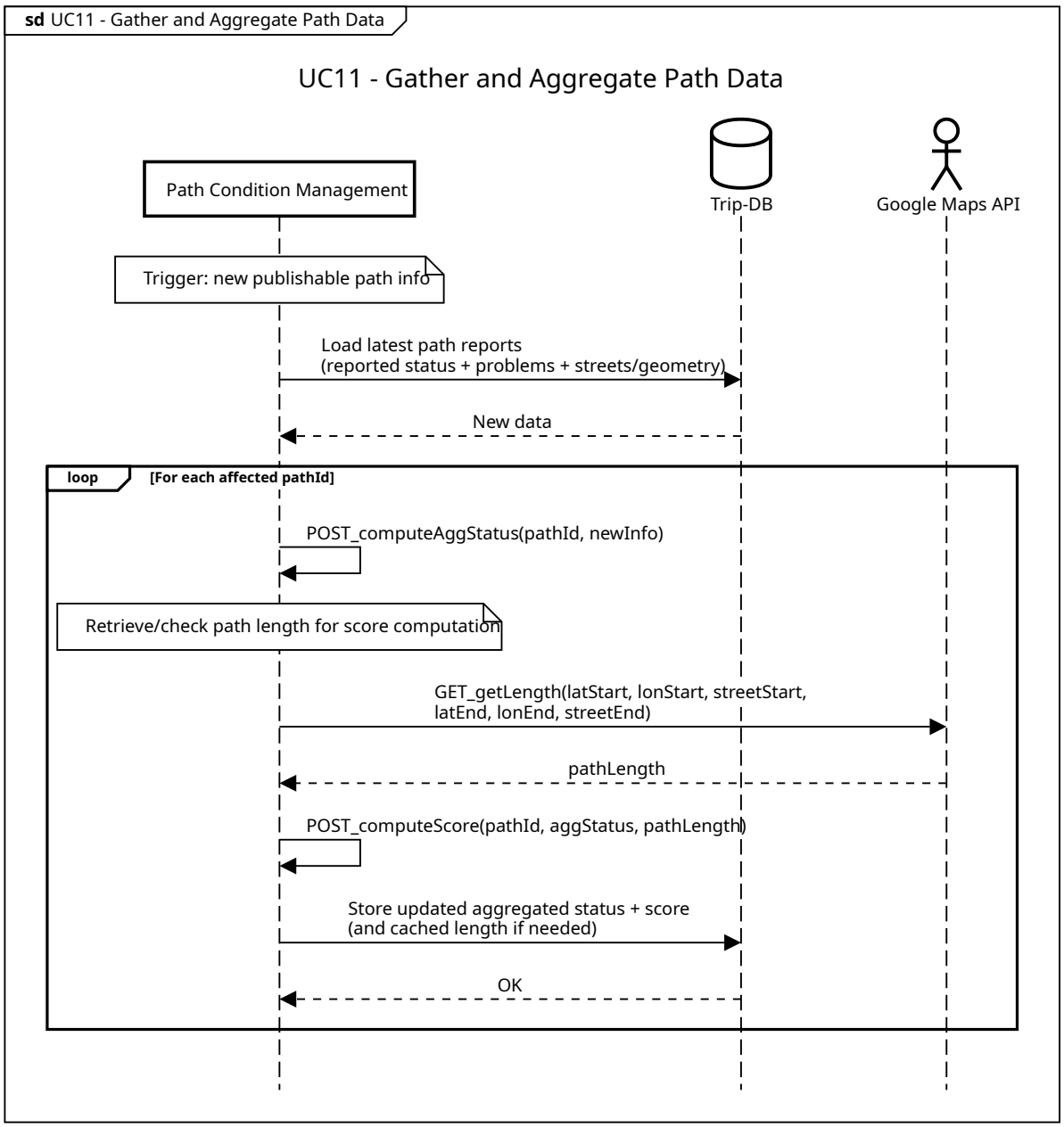
This sequence shows how a private path becomes publishable for the community. The Trip Recording & Publication microservice retrieves inferred privacy zones through `GET_getPrivacyZones` to sanitize GPS traces near sensitive locations. It then creates/updates the shared path information via the Path Condition Management microservice and publishes confirmed obstacles by reporting them as path problems, making the information available to all users.

[UC10] - Search Path



This diagram illustrates the search for bike paths between an origin and a destination. When textual addresses are provided, the system converts them into coordinates using `GET_geocode` via the Google Maps API. The Path Research microservice retrieves candidate paths, enriches them with aggregated status and active problems from internal components, ranks the results according to the path score, and returns them for map visualization.

[UC11] - Gather and Aggregate Path Data



This diagram represents an internal system task for data maintenance. Whenever new publishable information is stored, the Path Condition Management microservice aggregates recent reports to recompute the path status and score. Freshness and number of confirmations are considered when determining the aggregated status, while path effectiveness (including path length) contributes to score computation. Updated aggregated values are stored to keep the inventory consistent and reliable.

2.6 Selected architectural styles and patterns

Client-Server 3-Tier Architecture

BBP is designed according to a classic client-server 3-tier architecture in order to clearly separate presentation concerns from application logic and data management. The Presentation Tier

consists of the Mobile Application, which handles user interaction and sensor data acquisition, and the API Gateway, which represents the only entry point to the backend. The Application Tier hosts the core business logic, implemented as a set of independent microservices, while the Data Tier is responsible for persistent storage of user-related and operational trip and path data. This architectural style is great because it really improves maintainability, security and scalability by isolating responsibilities and controlling access to system resources.

Microservices Architecture

BBP adopts a microservices architecture to support scalability, fault isolation and independent evolution of system functionalities. Core services, such as User & Authentication Management, Accaunt Management, Trip Recording & Publication, Path Condition Management and Path Research, are deployed as autonomous components, each exposing well-defined RESTful interfaces. Requests are distributed across replicated instances through a load balancer, ensuring efficient resource utilization and consistent performance under varying workloads. This architectural style prevents system-wide failures and allows each service to be scaled or updated independently.

Circuit Breaker Pattern

To enhance system resilience and prevent cascading failures, BBP employs the Circuit Breaker pattern within the Application Tier. This pattern is particularly relevant for interactions with external services (Google Maps API and the Weather.com API) which may experience temporary unavailability or latency. By monitoring failures and temporarily blocking requests to unstable dependencies, the system maintains overall availability and enables graceful degradation while affected services recover.

State Pattern

The State Pattern is used to manage the lifecycle of key domain entities, such as Trips and Paths, whose behavior depends on their current state. For example, a trip may transition from *Recording* to *Completed* and finally to *Published*, while path information can move from *Private* to *Public*. This pattern allows the system to enforce valid user actions at each stage (e.g., preventing edits after publication) and to keep the related business logic structured and easily extensible.

Command Pattern

The Command Pattern is adopted to decouple user interactions in the Mobile Application from their corresponding execution logic within the backend microservices. By encapsulating operations, such as starting a trip, reporting a problem or uploading sensor data, into discrete command objects, the system simplifies request management and enhances extensibility. This pattern is particularly vital for handling the high volume of sensor data generated during cycling: GPS and accelerometer samples are buffered on the device and transmitted as a single "upload command," allowing the Application Tier to process them efficiently and asynchronously without impacting the user experience.

2.7 Other design decisions

API Gateway as Intermediary

The BBP Application Tier incorporates an API Gateway as the central intermediary between the client-side applications and the backend services. This component is responsible for orchestrating requests from the user interface to the internal microservices. By centralizing these

interactions, the API Gateway manages logging, caches frequently requested data, and conducts security checks before forwarding the processed responses back to the client. This architectural choice ensures that internal services remain decoupled from external network complexities and establishes a single point of control for rate-limiting, authentication and request transformation.

Layered Security and Firewalls

To ensure a robust defense against external threats, the system implements a multi-layered security architecture. In addition to the primary firewall that filters incoming traffic from mobile clients , a second firewall layer is strategically placed between the Application Tier and the external service providers. This configuration establishes a secure perimeter, preventing application servers from directly exposing themselves to untrusted external networks. Furthermore, the API Gateway enhances security by managing all external requests and coordinating with the User & Authentication Management microservice to verify tokens before allowing access to the core business logic.

Administrators Access

For system maintenance, debugging, and routine updates of the BBP servers, administrators are provided with a secure access method via Secure Shell (SSH). This protocol allows authorized personnel to establish an encrypted channel over unsecured networks to manage the microservices cluster and the Database Servers. Access is restricted to authorized users through the use of SSH keys, ensuring that administrative commands and file transfers can be executed remotely without compromising the integrity of the data tier or the availability of the application services.

Database Redundancy and RAID

To guarantee high availability and fault tolerance in the Data Tier, the system employs RAID (Redundant Array of Independent Disks) techniques on its database servers. This hardware-level decision ensures that the User Data & Profiles DB and the Trip & Path Data DB remain operational even in the event of a disk failure. This decision, combined with the replication of the application servers, ensures that the BBP platform satisfies the non-functional requirements for reliability and service continuity.

3 USER INTERFACE DESIGN

In this chapter, we present the User Interface of our mobile application. With respect to the RASD, particular emphasis is placed on user interactions and on the sequence of pages that define the overall **navigation flow**. To improve readability and facilitate understanding, the following legend is adopted:

- **Red circle labeled with a letter**: represents a user click on a specific element of the interface and the successive page.
- **Blue circle labeled with a letter**: denote the actual page, in which we are.
- **Red circles labeled with a letter followed by an asterisk**: indicate interactions that do not lead to a new page, but trigger a refresh of the current view, resulting in a minor modification of its content.

The button "About us" is not associated with a red-dot, because when the user clicks this button, he is directly redirected to the default web browser, which shows a web page containing more specific information about our program development.

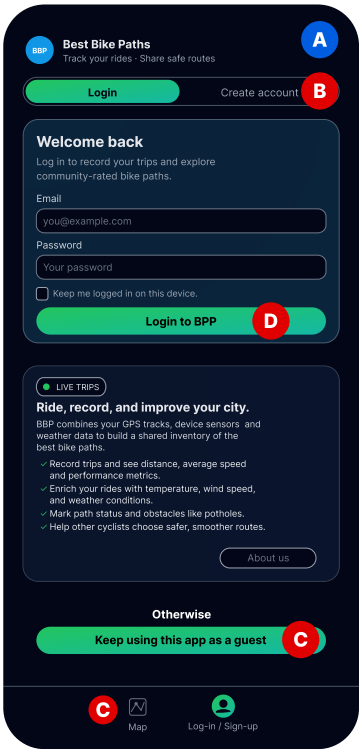


Figure 4: User login

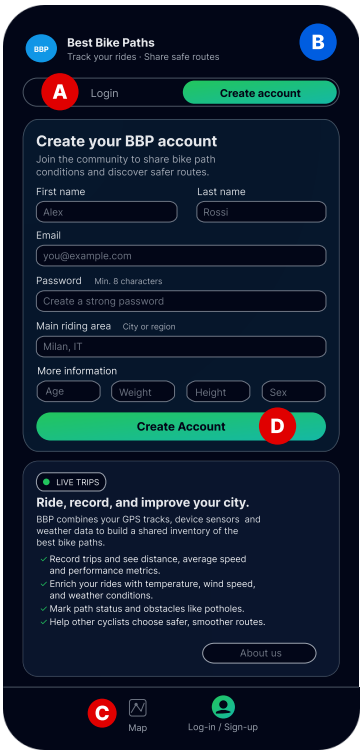


Figure 5: Account creation

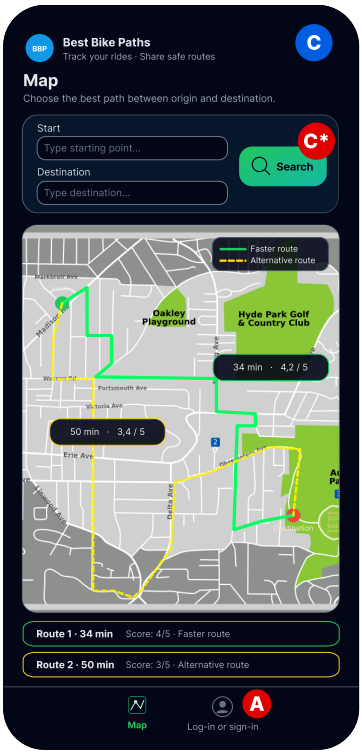


Figure 6: Map visualization (for non-registered or non-logged-in users)

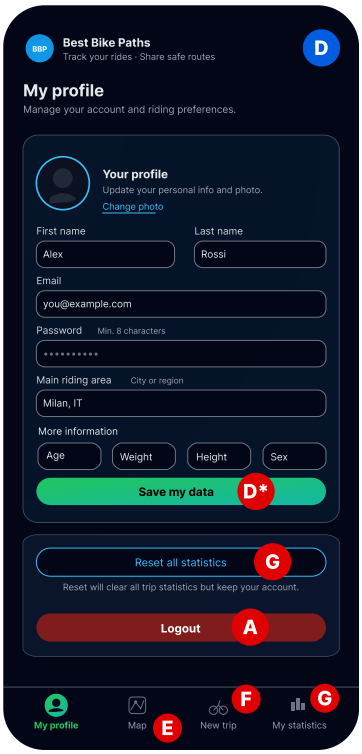


Figure 7: Profile edit and visualization

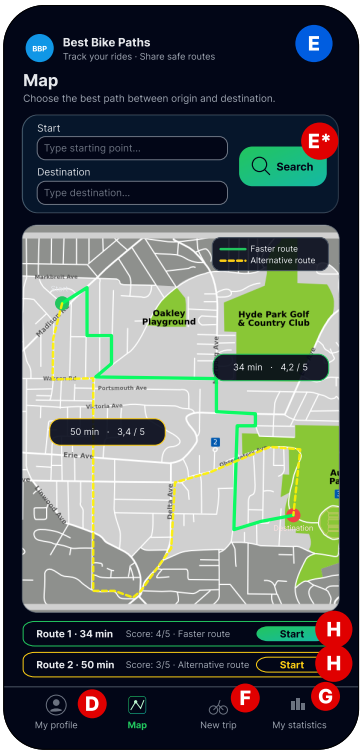


Figure 8: Map visualization (for logged-in users)

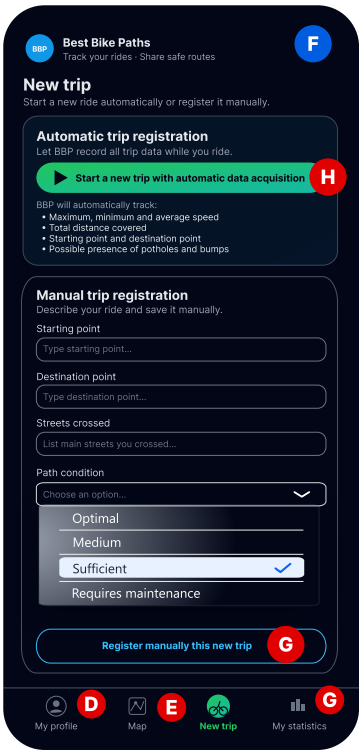


Figure 9: Start a new trip registration (in automatic mode or in manual mode)

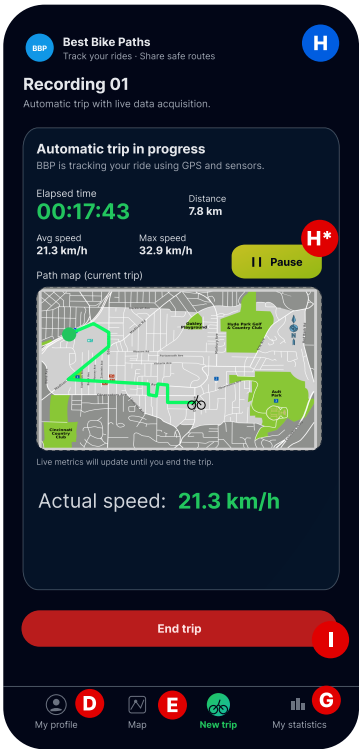


Figure 10: Recording a trip in automatic mode

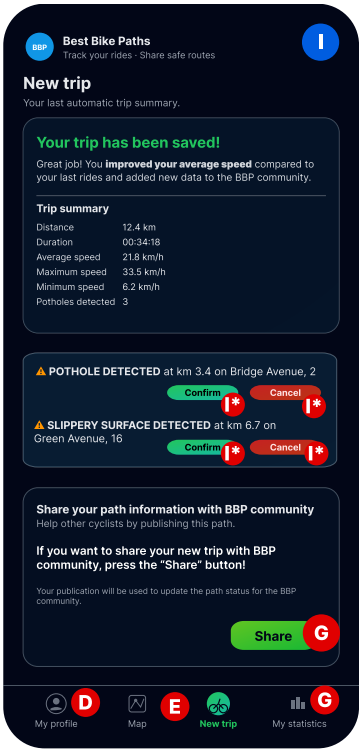


Figure 11: End of trip registration and publication



Figure 12: Visualization of personal statistics

4 REQUIREMENTS TRACEABILITY

The following Traceability Matrix maps each functional requirement to the architectural components responsible for its implementation. An **X** indicates that the component plays a primary role in fulfilling the requirement (e.g., it implements the logic, stores the data, or provides the external capability).

Req.	Description	App	API GW	Auth	Acc Mng	Trip	Cond	Search	U-DB	Trip-DB	Maps	Weather
R1	Create account	X	X	X					X			
R2	User log-in	X	X	X					X			
R3	Update profile	X	X		X				X			
R4	User log-out	X	X	X					X			
R5	Access control		X	X		X	X	X	X			
R6	Automatic trip recording (GPS/Sensors)	X	X			X				X		
R7	Statistics calculation		X			X				X		
R8	Weather data retrieval		X			X				X		X
R9	Trip storage		X			X				X		
R10	Manual path info insertion	X	X			X	X			X		
R11	Automatic path data gathering	X	X			X	X			X	X	
R12	Problem detection	X	X			X	X			X		
R13	Data publishing	X	X		X	X	X			X		
R14	Information aggregation		X				X			X		
R15	Path score calculation		X				X			X	X	
R16	Path status determination		X				X			X		
R17	Status update on new info		X				X			X		
R18	Problem resolution		X				X			X		
R19	Path search	X	X					X		X	X	
R20	Map visualization	X	X					X		X	X	
R21	Ranking display	X	X					X		X		
R22	View path details	X	X				X	X		X	X	
R23	Detected obstacles	X	X			X	X			X		

Table 1: Traceability Matrix: Requirements to Components

Legend:

- **App**: Mobile Application
- **API GW**: API Gateway
- **Auth**: User & Authentication Management
- **Acc Mng**: Account Management
- **Trip**: Trip Recording & Publication
- **Cond**: Path & Condition Management
- **Search**: Path Research
- **U-DB**: User Data & Profiles DB
- **Trip-DB**: Trip & Path Data DB
- **Maps**: Google Maps API
- **Weather**: Weather.com API

5 IMPLEMENTATION, INTEGRATION AND TEST PLAN

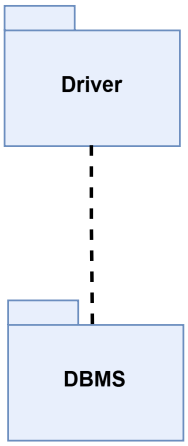
5.1 Overview

In this chapter, the Implementation, Integration and Test Plan for the *Best Bike Paths (BBP)* system is presented. The objective of this plan is to describe a structured strategy for building the system incrementally, ensuring reliability, modularity and testability through the development process.
The implementation and integration activities will follow a **Bottom-Up integration strategy**. This approach starts from the lowest-level components of the system architecture, which are independent modules with minimal dependencies, and progressively integrates them into higher-level subsystems.
Each module is initially tested in isolation using dedicated drivers. Once validated, the module replaces its corresponding driver and is integrated with other tested components. This incremental process allows early fault detection, simplifies debugging, and supports parallel development. By adopting this strategy, BBP ensures that core services such as data persistence, sensor data handling and path aggregation are robust before integrating user-facing functionalities and external services.

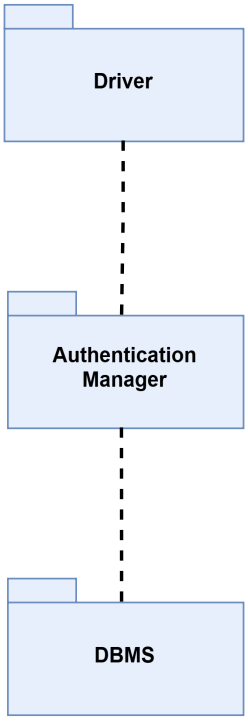
5.2 Implementation Plan

The implementation plan follows a Bottom-Up approach, starting from simple services and progressively moving toward complex system functionalities and user interactions.

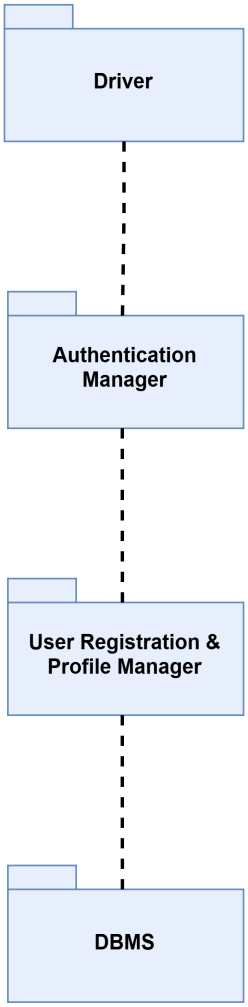
1. **Database Management System (DBMS)** The first component to be implemented is the DBMS, which represents the persistent storage layer of the BBP system. It stores user accounts, trips, paths, problems, scores and aggregated path information. Implementing the database first ensures a stable backend for all data-dependent modules.



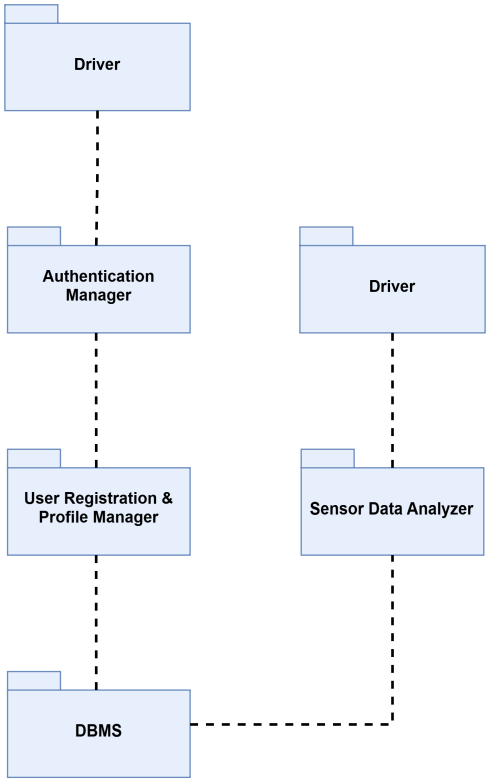
2. **Authentication Service** Once the DBMS is available, the Authentication Manager is implemented. This module handles secure login, session management and access control for registered users. Its correct functioning is essential for protecting all registered-only features of the system and also private information.



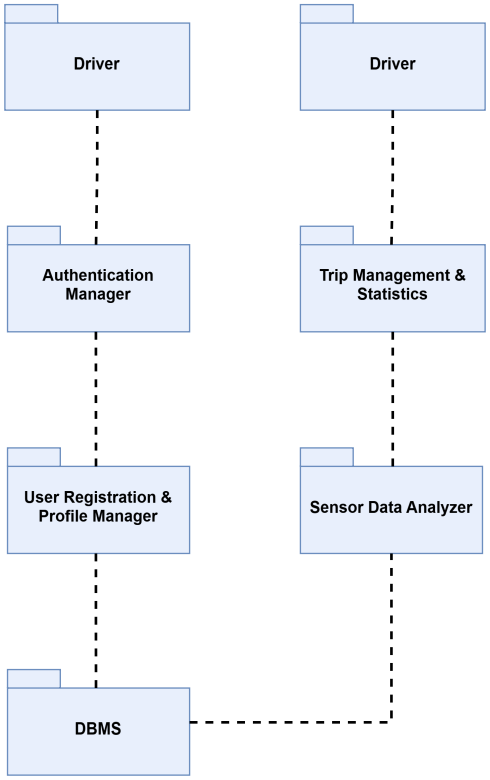
3. **User Registration & Profile Management** This step includes the implementation of user registration services and profile update functionalities. These components depend on both the DBMS and Authentication Services and allow users to create accounts, modify personal data, and manage preferences.



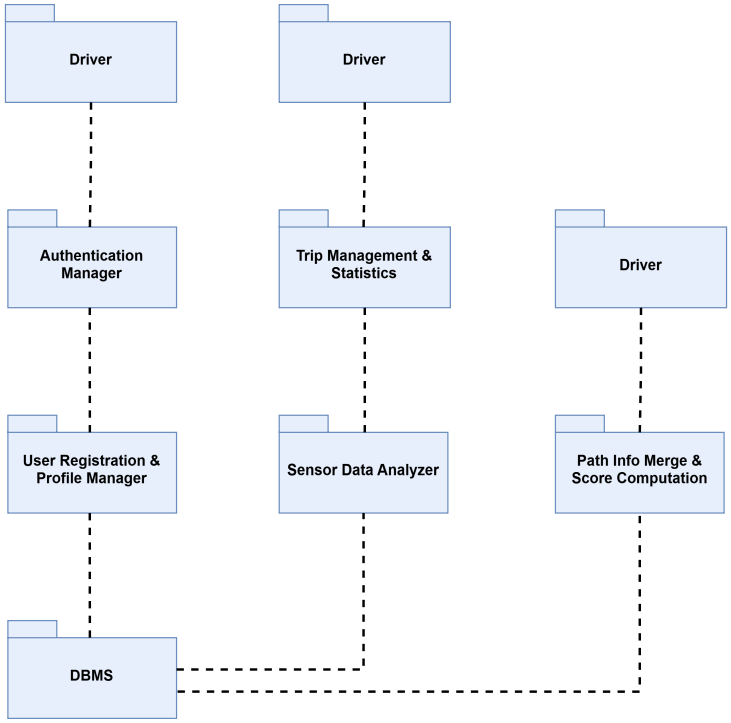
4. **Sensor Data Analyzer** The Sensor Data Analysis module is responsible for processing and analyzing GPS, accelerometer, and gyroscope data received from the mobile client in order to automatically detect path-related issues. The module operates independently from user management and authentication services and is tested in isolation using simulated sensor datasets before being integrated with the rest of the backend system.



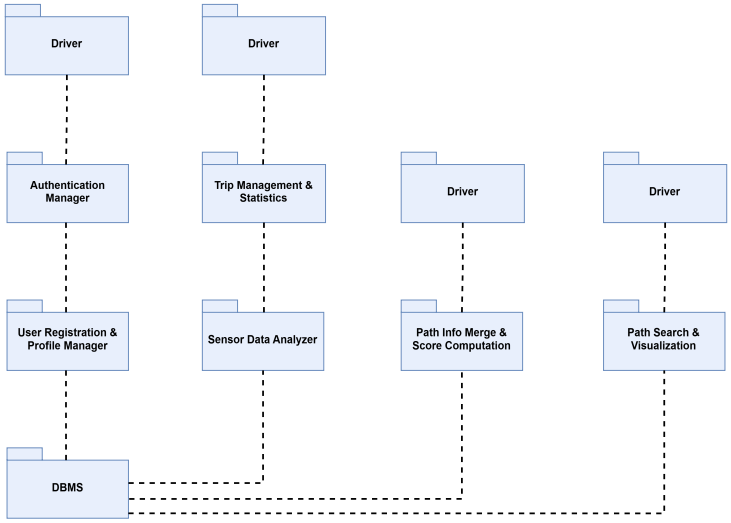
5. **Trip Management and Statistics Module** After sensor data handling is validated, the Trip Manager is implemented. This component computes trip statistics (distance, duration, speed), and stores trips in the database. It relies on previously implemented sensor and persistence layers.



6. **Path Information Merge and Score Computation Module** This module aggregates user-contributed information about bike paths, evaluates path status, and computes path scores. It integrates data from multiple users, considering freshness and confirmation rules, and updates the shared path inventory.



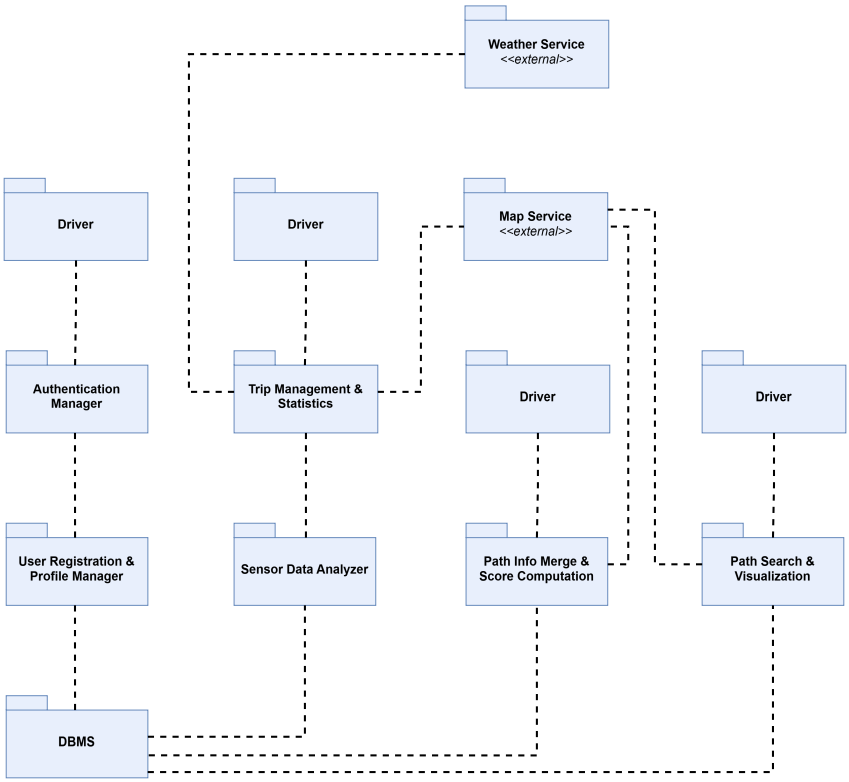
7. **Path Search and Visualization Module** This component enables registered and non-registered users to search for paths between two locations, visualize them on a map, and compare scores. It relies on the mapping service, path aggregation module and scoring logic.



8. **External Services Integration (Maps and Weather Services)** At this stage, external services are integrated:

   - Mapping services for route computation and visualization;
   - Weather services for enriching trip data.

   Before connecting to real external services, these components are tested using simulated API responses. This simulation allows verifying that the system correctly finds, processes, and returns the expected results based on the internal data structures.



5.3 System Testing Plan

Once all components have been implemented and integrated, system testing activities are performed to verify that the BBP system behaves correctly as a whole and satisfies the requirements defined in the RASD.

The testing process includes the following phases:

- **Unit Testing** Each module is tested independently using mock data (manually simulated) and test drivers. This phase verifies the correctness of individual components such as sensor data processing, score computation and database access logic.
- **Integration Testing** Interactions between integrated modules are tested to ensure correct data flow and interface compatibility. The most important scenarios to test are:
  - Sensor Data Acquisition interacting with Trip Management
  - Path Information Merge and Score Computation Module interacting with Path Search
  - Trip saving triggering Weather Data retrieval and Map Service
- **Functional Testing** Functional testing validates that all system functionalities satisfy the requirements and use cases defined in the RASD. Each use case is executed end-to-end to ensure correct behavior from the user's perspective, in order to ensure the whole functionality of the BBP application.

- **Performance Testing** Performance tests evaluate system responsiveness, scalability, and resource usage under realistic and peak conditions. Particular attention is given to:
  - Path search response time
  - Concurrent users accessing the backend
  - Battery consumption during automatic trip recording
  - Problems correctly detected

This structured testing approach ensures that the BBP system is reliable, scalable, and compliant with the specified functional and non-functional requirements.

6 EFFORT SPENT

The following table summarizes the effort spent by each team member for the preparation of the main chapters of this document, expressed in working days.

Team Member	Chapter 1	Chapter 2	Chapter 3	Chapter 4	Chapter 5
Andrea Rubagotti	1	3	1	1	1
Maria Concetta Santagata	1	3	1	1	1
Samuele Leanza	0	0	0	0	0

Table 2: Effort spent (in days) by each team member per document chapter

7 REFERENCES

The assignment for this document and all the information included refer to the following documentation:

References

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the WeBeep page of the Software Engineering II course [`https://webeep.polimi.it/`].

Used Tools

- **OpenAI** - *ChatGPT*. Used as an auxiliary tool for drafting and refining sections of this document.
- **GitHub** for project versioning and sharing.
- **LATEX and Overleaf** as editor for writing this document.
- **sequencediagram.org** for the sequence diagrams.
- **draw.io** for all the remaining diagrams' design.
- **Figma** for the UI's design.
- **Google Documents** for collaborative writing, notes and reasoning.